

Milestone 5 Report

Smart MCQ Solver Challenge — Ensemble Inference, Test-Time Augmentation & Confidence Analysis

This milestone covers combining two fine-tuned models (DeBERTa-v3-small and RoBERTa-base) via probability ensembling, analyzing where they agree and disagree, applying Test-Time Augmentation (TTA), measuring confidence gains from ensembling, and computing MAP@3 on a validation subset.

Section 1 — Single Model Inference (Q1)

Q1. DeBERTa top-1 option and probability for row 25

Built a combined prompt string from the row 25 question and all 5 option texts, tokenized it with the DeBERTa tokenizer (max_length=256, truncation), passed through the model, applied softmax to the 5 output logits, and identified the option with the highest probability. The get_probs() helper handles the full tokenize → forward pass → softmax chain and returns a 5-element numpy array.

↳ The prompt is formatted as '[question text] A: [opt_A] B: [opt_B] C: [opt_C] D: [opt_D] E: [opt_E]' — concatenating all options into one sequence classification input rather than using the [batch, 5, seq_len] multiple-choice format from Milestone 4. This is simpler for inference-only use and avoids the reshape overhead.

Section 2 — Probability Ensembling (Q2–Q4)

Q2. Simple average ensemble — top-1 option for row 25

Ran RoBERTa inference on the same row 25 prompt using the same get_probs() helper. Averaged the two 5-element probability arrays element-wise: $\text{avg} = (\text{P_deberta} + \text{P_roberta}) / 2$. Since both outputs are already softmaxed to sum to 1.0, their average also sums to 1.0. Took the argmax of the averaged array and mapped back to the option letter.

↳ Simple averaging treats both models as equally reliable. It tends to reduce variance — options that one model is overconfident about get pulled back toward the average, and options that both models agree on get reinforced. Works well when both models have similar accuracy.

Q3. Weighted ensemble (0.70 DeBERTa / 0.30 RoBERTa) — top-1 option for row 25

Applied $\text{P_final} = 0.70 \times \text{P_deberta} + 0.30 \times \text{P_roberta}$ element-wise. DeBERTa gets a larger weight because it generally outperforms RoBERTa on NLU tasks due to its disentangled attention mechanism, and in this challenge it's the larger/better model. Took the argmax of the weighted sum.

↳ Weighted ensembling lets you bias toward whichever model you trust more based on prior validation performance. If DeBERTa scored 0.64 and RoBERTa scored 0.55 on the leaderboard, a 0.70/0.30 split is a reasonable starting point — though the optimal split can be found by sweeping weights on a held-out validation set.

Q4. Top-3 prediction string for row 25 in Kaggle submission format

Used the weighted ensemble probabilities from Q3. Sorted all 5 option indices by probability descending with np.argsort(-probs), took the first 3, mapped each index to its letter via LABEL_MAP (0=A, 1=B, 2=C, 3=D, 4=E), and joined with spaces. The result is a string like 'C A E' matching the required submission format.

↳ The top3_string() helper encapsulates this: argsort → slice [:3] → map to letters → join. Used consistently for both DeBERTa-only and ensemble predictions throughout the rest of the milestone.

Section 3 — Full Test Set Submission (Q5)

Q5. Number of prediction rows in submission.csv

Looped over all rows of test.csv, ran both models with get_probs(), applied the 0.70/0.30 weighted blend, took top-3, and saved to submission.csv with columns ID and Prediction. The number of prediction rows equals the number of rows in test.csv (excluding the header). Confirmed with len(submission).

↳ Running both models on every test row is the slow step — two forward passes per row. Batching would speed this up substantially, but the single-row loop keeps the logic simple and is fast enough on GPU for a competition-sized test set.

Section 4 — Test-Time Augmentation (Q6)

Q6. Rows with different Top-1 after TTA on first 50 test rows

For each of the first 50 test rows, ran DeBERTa twice: once on the original prompt string and once on the same string with 'Answer the following multiple-choice question carefully: ' prepended. Averaged the two probability arrays, then checked whether the Top-1 letter changed compared to the original-only pass. Counted how many rows flipped.

↳ TTA exploits the fact that LLMs are sensitive to prompt phrasing — a small instruction prefix can shift probability mass. Averaging the two passes smooths out that sensitivity. If a flip happens, it means the model was uncertain enough that the phrasing change was decisive — those are exactly the rows where TTA is most valuable.

Section 5 — Model Comparison & Confidence Analysis (Q7–Q9)

Q7. Rows with different Top-1 between DeBERTa and Weighted Ensemble (first 100 test rows)

For each of the first 100 test rows, ran both models, computed the weighted ensemble, and compared Top-1 letters. Counted disagreements. This directly measures how often RoBERTa's 0.30 weight is enough to flip DeBERTa's top prediction — which only happens when DeBERTa was not very confident to begin with.

↳ A low count here means the ensemble rarely overrides DeBERTa — the 0.70/0.30 weighting is conservative enough that RoBERTa is mostly just fine-tuning probabilities rather than changing the winner. A high count would suggest the two models have more disagreements and the weights need revisiting.

Q8. Rows with positive confidence gain from ensembling (first 100 test rows)

For each row, computed DeBERTa's max probability (confidence) and the ensemble's max probability, then checked whether ensemble confidence > DeBERTa confidence. Confidence gain = $\max(P_{\text{ensemble}}) - \max(P_{\text{deberta}})$. Counted rows where this difference is strictly positive.

↳ Ensembling can increase or decrease the top confidence depending on whether RoBERTa agrees or disagrees with DeBERTa's top pick. If RoBERTa agrees, its 0.30 weight adds probability mass to the same option — confidence goes up. If it disagrees, it pulls probability toward a different option — confidence goes down. The count of positive gains tells you how often RoBERTa is reinforcing DeBERTa.

Q9. Rows where ordered Top-3 string differs between DeBERTa and Ensemble (first 100 test rows)

Compared the full Top-3 prediction string (e.g. 'A C D' vs 'A D C') for DeBERTa-only and weighted ensemble for each of the first 100 test rows. Any difference in the string — whether a letter swapped position or a different letter entered the top 3 — counts as a change. This is a broader measure than Q7 (Top-1 only) and captures subtler reranking effects.

↳ Q9 count $\geq$ Q7 count always: every Top-1 flip is also a Top-3 string change, but the reverse isn't true — the Top-1 can stay the same while 2nd and 3rd place swap. Q9 – Q7 gives you the number of rows where ensembling only reordered the bottom of the top-3 without changing the winner.

Section 6 — MAP@3 Validation (Q10)

Q10. MAP@3 on first 100 validation rows using weighted ensemble

Ran the weighted ensemble pipeline on the first 100 rows of train.csv (where answers are known). For each row, took the Top-3 prediction string, checked the rank of the true answer within those 3 (1/1, 1/2, or 1/3), or 0 if not present. Averaged across all 100 rows and rounded to 4 decimal places.

↳ Using train.csv rows as a validation proxy gives an honest MAP@3 estimate since these rows have known answers. The score won't exactly match the leaderboard (different distribution of questions) but is a reliable signal for comparing different ensemble configurations.